AULA 17

Fechar o item 2, a câmera que não é a sua, e a rede que decora

Etapa 9 · terça, 10/11/2026 · sala SJ205 · Prof. Dácio Moreira de Souza

Acertar 90% respondendo sempre a mesma coisa não é aprender.

É a quarta vez no semestre que uma métrica mede o instrumento e não o mundo: o IoU alto do pior detector, a nuvem mais concentrada do robô mais perdido, a trajetória mais longa do robô parado. Hoje é a acurácia da rede que decora.

Cenas dos últimos episódios: onde paramos

- O diagnóstico do Nav2 e os três casos rodaram ao vivo: sem pose, intruso, gerenciador parado
- A primeira linha de "Onde olhar" é a que conta; as outras costumam ser eco dela
- No Humble não há nó de árvore que meça distância a obstáculo: quem diz é você
- O bloco da sessão gravada foi o que cedeu, como combinado — e ele abre a aula de hoje
- A régua sintética já estava pronta: 25,52 m mandados, 49,03 m somando o /amcl_pose

Os conceitos de antes que entram hoje

- Cancelar e reenviar um objetivo (Aula 8): é o que o vigia faz, sem o Nav2 saber dele
- cor -> borda -> reta, com os limiares no topo do arquivo (Aula 13): hoje sobre outra câmera
- Métrica declarada antes de medir (Aula 9): vale para limiar de faixa e para acurácia
- A linha de base: quanto acerta quem não olha a imagem e responde sempre o mesmo
- Versão desalinhada falha calada: hoje são três Pythons, e cada um tem um motivo

O que você sai sabendo hoje

- Gravar uma sessão, tocá-la sem o robô e tirar dela a tabela e a figura do item 2
- Dizer onde mora a decisão de interromper — e o que a régua diz quando ela trapaceia
- Subir o MetaDrive, capturar quadros, e saber o que a cena tem e o que ela não tem
- Ajustar um detector a uma câmera nova, e declarar o ajuste em vez de escondê-lo
- Treinar a mesma rede duas vezes e saber qual número da tabela merece confiança

E o que cada uma vale no SEU projeto

- O bag: a análise roda no ônibus, sem robô, sem simulador e sem WSL2 esquentando
- A decisão de interromper: é arquitetura, e é a pergunta certa da arguição
- A cena sintética: o item 3 deixa de depender de você ter uma câmera boa
- O limiar declarado: o relatório explica o detector em vez de pedir desculpas por ele
- A matriz de confusão: a frase do relatório que separa a rede que aprende da que decora
- As datas de cada entrega estão na página e em "Gates de cada TP" — consulte, não decore

Como a aula funciona hoje

- Eu demonstro no meu ambiente, e vocês anotam o que vão procurar na tela de vocês
- Três casos ao vivo: o detector que acha zero, a normalização que engana, a classe que some
- O resto foi medido antes, e está na página com os números — eu mostro, não repito
- O que é para rodar no SEU ambiente está na tarefa da semana, em ordem, com terminais
- O MetaDrive é lento: 500 quadros são uns 23 minutos. Por isso ele entra na tarefa, não aqui

Onde está o material

- A página da Aula 17: o que eu mostrar hoje, com os comandos, e a tarefa da semana
- Os slides em PDF, na mesma página
- aula16-arvore-bag: o vigia, o gravar_pose e a régua da sessão — da semana passada
- aula17-metadrive: a captura, a medição das faixas e as formas
- aula17-cnn: o sintético, a rotulagem, o treino das duas rodadas e as curvas
- O bloco de download abre a página e o trabalho em casa — não é para agora

BLOCO 1

Fechar o item 2: a interrupção e a sessão gravada

Terminal A: o sistema, com a pose de largada

```
export PB_USER=dacioms
export PB_DIR="$HOME/projeto-pb-$PB_USER" && export PB_WS="$PB_DIR/ros2_ws"
source /opt/ros/humble/setup.bash
source "$PB_WS/install/setup.bash"
ros2 launch aula15_nav2 nav2.launch.py rviz:=true pose_inicial_auto:=true

# a largada: (2,0; 4,0), olhando para a porta da sala 2
```

O mesmo sistema da semana passada, agora com a pose vinda do launch: sem clique, a sessão começa igual toda vez — e é isso que deixa o bag comparável.

Onde mora a decisão: num nó seu, fora da árvore

- O vigia assina o /scan e olha um leque de ±60° à frente do robô
- Abaixo do limiar: cancela o objetivo — a Aula 8, operação pública da ação
- Espera 2 s, reenvia o MESMO objetivo, e não olha o laser por 1,5 s de carência
- Publica /estado_vigia e /dist_min: é com eles que o bag registra POR QUÊ parou
- O Nav2 não sabe que o vigia existe — e não precisa saber
- Honestidade: ele rodou pela primeira vez no meu ensaio de ontem, não antes

Terminal B: o vigia, e o objetivo pela porta

```
# Terminal B, novo: as variaveis e os dois source, como no Terminal A
ros2 run aula16_arvore vigia --ros-args \
  -p objetivo:="[9.0, 2.2, -1.57]" -p limiar_m:=0.75

# o que deve aparecer (o vigia rodou pela primeira vez no ensaio):
#  enviando objetivo (9.00, 2.20, -1.57)
#  obstaculo a 0.7x m (< 0.75 m): cancelando o objetivo
#  cancelamento aceito (...); esperando 2.0 s
#  espera cumprida; retomando o MESMO objetivo
```

No mundo da Aula 11 nada entra na frente do robô. Para o portão disparar, subi o limiar para 0,75 m e mandei o robô pela porta: o batente faz o papel do obstáculo.

Terminal C: gravar a viagem inteira

```
# Terminal C, novo (variaveis e os dois source), ANTES de o vigia mandar o goal
mkdir -p "$PB_DIR/bags" && cd "$PB_DIR/bags" && rm -rf demo-aula17
ros2 bag record -o demo-aula17 /odom /amcl_pose /scan /cmd_vel \
  /estado_vigia /dist_min /tf /tf_static

# do envio ate a chegada; Ctrl-C quando o vigia disser "objetivo cumprido"
```

Tópicos nomeados, nunca -a: com os costmaps o bag passa de um giga. E o /estado_vigia na lista é o que deixa a figura marcar as interrupções.

O bag info, e a linha que não se corrige depois

```
ros2 bag info demo-aula17

#  Topic: /tf_static     | Type: tf2_msgs/msg/TFMessage | Count: 1 | ...
#  Topic: /estado_vigia  | Type: std_msgs/msg/String    | Count: ...
```

O /tf_static sai UMA vez, quando o sistema sobe. Fora da lista, a reprodução é um laser flutuando sem corpo — e você descobre quando não há mais como regravar.

Tocar sem o robô: play --clock, e o CSV saindo

```
# Terminal A, depois do Ctrl-C no sistema e no vigia (Terminal B):
cd "$PB_DIR/aula16-arvore-bag"
ros2 launch aula16_arvore analise.launch.py saida:=demo-aula17.csv

# Terminal C, na pasta dos bags:
ros2 bag play demo-aula17 --clock
```

O analise.launch.py sobe só o gravar_pose, com use_sim_time:=true. Sem o --clock no play, o nó espera um relógio que nunca chega — e não grava nada, sem uma linha de erro.

A régua sobre o bag de verdade

```
# Terminal B, quando o play terminar e o Terminal A levar Ctrl-C:
cd "$PB_DIR/aula16-arvore-bag"
python3 medir_sessao.py demo-aula17.csv
python3 plotar_trajetoria.py demo-aula17.csv --saida demo-aula17.png

# o que a regua deve dizer, com o limiar de 0,75 m:
#    ATENCAO: houve interrupcao, mas o /scan nunca ficou abaixo de 0.50 m.
#    O que disparou nao foi o portao.
```

A mesma régua da sessão sintética, sem mudar uma linha. E ela pega o meu atalho: o limiar do enunciado está declarado no topo dela, e eu disparei a 0,75 m.

O que a sessão sintética já ensinou, e vale para o seu bag

- Três distâncias: 25,52 m mandados; 26,29 m no /odom; 49,03 m somando o /amcl_pose
- A terceira é o tremor virando caminho: não cite distância a partir do /amcl_pose
- O vão de 0,75 m entre /odom e /amcl_pose tem nome: é o que a map -> odom corrige
- As esperas de 2,00 s e 2,10 s: tolerância declarada antes, ± 0,25 s, como na Aula 9
- A régua não sabe de onde o CSV veio: sintético ou bag, a conta é a mesma

Esperar 2 s só resolve obstáculo que sai sozinho.

Contra uma caixa parada, o vigia vira cancelar, esperar, tentar, cancelar — e a régua acusa REINCIDENCIA. No mundo mínimo nada entra e sai da frente do robô; no Fortress dá para pôr e tirar uma caixa. Qualquer que seja o seu, ele vai declarado no relatório.

O item 2 no relatório: o que se declara

- Em que mundo a sessão foi gravada: o mínimo da Aula 11, ou o Gazebo Fortress
- Onde mora a decisão de interromper: num nó seu (cliente) ou num nó de árvore em C++
- O limiar que disparou de verdade, lido na régua — e não o que estava no enunciado
- Os tópicos gravados, e o que ficou de fora do bag e por quê
- A tabela e a figura saídas do SEU bag, com a distância citada pela conta certa

BLOCO 2

A câmera sintética: o MetaDrive, as faixas e as formas

A instalação que funcionou

```
cd "$PB_DIR"
uv venv --python 3.10 --system-site-packages .venv-metadrive
uv pip install --python .venv-metadrive/bin/python "metadrive-simulator==0.4.3"

# a conferencia do proprio pacote, que liga a camera:
cd "$PB_DIR/aula17-metadrive"
"$PB_DIR/.venv-metadrive/bin/python" -m metadrive.examples.verify_headless_installation \
  --camera rgb
#   Headless mode Offscreen render launched successfully!
```

O --python 3.10 é obrigatório: o Panda3D do MetaDrive 0.4.3 não tem wheel para 3.12 nem 3.13. Sem servidor X, o erro é um set_shader em None — o xvfb-run resolve.

A captura: lenta, e determinística

```
cd "$PB_DIR/aula17-metadrive"
"$PB_DIR/.venv-metadrive/bin/python" capturar_frames.py --frames 20 --saida quadros-aula

#  forma da imagem : (240, 320, 3), uint8, canais BGR
#  quadros gravados: 20 em quadros-aula (+ estado.csv)
#  na nuvem: 0.37 quadros/s -> 500 quadros, uns 23 minutos
```

Quem dirige é o piloto IDM do próprio MetaDrive: mesma semente, mesmo estado.csv. A imagem já sai BGR, com quatro eixos na observação — não inverta nada.

O que a cena tem, e o que ela NÃO tem

- NÃO há placa de trânsito no MetaDrive, nem semáforo no mapa procedural
- Há cones, um triângulo de advertência, uma barreira e veículos de cinco tamanhos
- O piloto IDM mantém distância: 54 via_livre, 0 obstáculo e 6 cruzamento em 60 quadros
- O enunciado diz "placas": a resposta honesta é trocar por cone e veículo, e dizer
- Inventar uma placa, ou baixar o limiar até aparecer um círculo, não é resposta

Caso 1: o detector da Aula 13, sem mudar nada

```
cd "$PB_DIR/aula17-metadrive"
python3 ../aula13-veicular/faixas.py --quadro amostra/quadro020.png --etapas /tmp/et-a13

#  faixa_esq   : NAO ACHADA
#  faixa_dir   : NAO ACHADA
#  etapa 1 cor :      0 px na ROI  (esq 0 / dir 0 / longe 0 / amarelo 0)
#  etapa 2 bord:      0 px de borda
```

A máscara de cor sai vazia: a tinta do MetaDrive para em V = 149, e o limiar da Aula 13 é 150. Um ponto de diferença. A etapa que quebra é a primeira, e as outras são eco.

As mesmas funções, com o limiar da fonte

```
python3 medir_faixas.py --pasta amostra --sobreposicao /tmp/faixas.png --quadro 4

#  quadro          PADRAO (A13)    AJUSTADA
#  quadro020.png     0 seg  . .     14 seg  E D
#  quadro055.png     0 seg  . .      0 seg  . .
#  PADRAO  : 12 quadros | ... | esq 0, dir 0, ambas 0
#  AJUSTADA: 12 quadros | ... | esq 11, dir 11, ambas 11
```

Ampliar 3x, branco a partir de V = 130, Canny (20, 60): declarados no topo, antes de medir. Nos 60 quadros: zero contra 51. O 055 é o cruzamento — lá não há tinta.

Limiar é da fonte de imagem, e se declara.

O detector da Aula 13 não está errado: ele foi afinado numa cena, e nada viaja sozinho para outra. O relatório diz qual configuração, para qual câmera e por quê — e continua sem verdade para medir erro: a tabela conta quantas vezes achou, não se achou certo.

Círculos e cantos: o que se escreve no relatório

- Hough Circle com param2 = 30: zero círculos — e é o número certo, porque não há placa
- Com param2 = 15 aparecem dezenas, a maioria no céu: falso positivo por construção
- Harris: de 30 a 180 cantos por quadro — o limiar é relativo e acha canto em qualquer imagem
- Os cantos que importam: quinas de veículo, base de cone, ponta de tracejado
- Para provar o detector, desenhe a forma num quadro real, em posição conhecida

BLOCO 3

A rede que decora

O problema começa no conjunto, não na rede

- Com o piloto IDM, 60 quadros saíram 54 : 0 : 6 — via livre, obstáculo, cruzamento
- A classe obstáculo não aparece sozinha: ela tem de ser provocada
- Mais tráfego, mais acidentes, cinco sementes de cem quadros — e dizer o que mudou
- Uma semente é uma estrada: 500 quadros dela são 500 vistas do mesmo trecho
- Por isso os números de hoje vêm de imagens sintéticas, equilibradas, 167/167/166

Rotular: a sugestão, a revisão, e o círculo vicioso

- O estado.csv sugere o rótulo por geometria do simulador, não por visão
- Ele chama de cruzamento os metros ANTES do cruzamento, e não vê o carro escondido
- Revisar quadro a quadro, e contar quantos você corrigiu: é a evidência de rotulagem
- Zero correções em 500 quadros é suspeito, não é mérito
- Rotular com a própria rede é circular: a matriz passa a medir a rede concordando consigo

Um venv à parte, sem o sistema: a exceção

```
cd "$PB_DIR"
uv venv --python 3.10 .venv-cnn
uv pip install --python .venv-cnn/bin/python \
  tensorflow-cpu opencv-python-headless matplotlib scikit-learn

"$PB_DIR/.venv-cnn/bin/python" -c "import tensorflow as tf, keras; print(tf.__version__)"
#   2.21.0
```

Sem --system-site-packages, só aqui: o TensorFlow traz NumPy 2, e um cv2 do apt importado ao lado dele morre com _ARRAY_API not found. Este venv nunca importa ROS 2.

Duas rodadas, a mesma semente

```
cd "$PB_DIR/aula17-cnn"
"$PB_DIR/.venv-cnn/bin/python" gerar_sintetico.py --saida sintetico
"$PB_DIR/.venv-cnn/bin/python" treinar.py --pasta sintetico --saida resultado
"$PB_DIR/.venv-cnn/bin/python" curvas.py --pasta resultado --saida resultado/curvas.png

# a mesma divisao, os mesmos pesos iniciais, a mesma ordem dos lotes;
# a unica diferenca: rotacao, zoom e flip na rodada "com"
```

Uns 45 segundos por rodada na CPU. Comparar com outra semente é comparar sorte: com 100 imagens de validação, cada imagem vale um ponto.

A tabela medida

```
linha de base: responder SEMPRE "via_livre" acerta 0.330 da validacao

rodada  acc_val  acc_treino  acc_val_pert
sem       0.980       0.913         0.910
com       0.930       0.678         0.940

matriz sem: 2 obstaculos chamados de via livre    <- o erro perigoso
matriz com: 7 vias livres chamadas de obstaculo   <- o carro que freia a toa
```

O augmentation piorou 5 pontos na validação normal — que não tem imagem girada. Na perturbada, a ordem se inverte. Mesma acurácia, comportamentos opostos no carro.

Três imagens em cem não sustentam conclusão.

A diferença na validação perturbada é 0,910 contra 0,940: três imagens. É a direção que a teoria prevê, e é exatamente isso que se escreve — a direção, e que ela não basta. E a última época sozinha é frágil: a rodada com augmentation fez 1,000 na 19 e 0,930 na 20.

Caso 2: a BatchNormalization com o padrão do Keras

```
"$PB_DIR/.venv-cnn/bin/python" treinar.py --pasta sintetico --bn-momento 0.99 \
  --saida resultado-bn099

rodada  acc_val  acc_treino
sem       0.330       0.913      <- o treino aprende; a validacao responde uma classe so

matriz sem:  via_livre 33 0 0 | obstaculo 34 0 0 | cruzamento 33 0 0
```

Treino com a média do lote; inferência com a média móvel. Com 260 passos e momentum 0,99, 7% dela ainda é o valor inicial. Com 0,9, a validação vai a 0,98.

Caso 3: 54 : 0 : 6, e a rede que nunca diz cruzamento

```
"$PB_DIR/.venv-cnn/bin/python" gerar_sintetico.py --proporcao 54:0:6 --saida desequilibrado
"$PB_DIR/.venv-cnn/bin/python" treinar.py --pasta desequilibrado --bn-momento 0.99 \
  --saida resultado-deseq

linha de base: responder SEMPRE "via_livre" acerta 0.900 da validacao
sem       0.900   (treino 1.000)    pior recall: cruzamento 0.000

                via_livre  obstaculo  cruzamento   <- previsto
  via_livre             90          0           0
  obstaculo              0          0           0
  cruzamento            10          0           0
```

Noventa por cento, e a coluna cruzamento vazia. Quem olhar só a acurácia entrega isto como resultado — e é a linha de base, nada mais.

Por que a matriz, e não só a acurácia

- A acurácia mede a proporção das classes antes de medir a rede
- A linha de base diz quanto acerta quem nem olha: 0,330 equilibrado, 0,900 no 54 : 0 : 6
- O recall por classe e a coluna vazia da matriz são o que denuncia a rede que decora
- IoU, nuvem, trajetória, acurácia: quatro vezes a métrica mediu o instrumento
- No relatório: a contagem por classe, a linha de base, a acurácia, e a matriz por último

A rede sadia no 54 : 0 : 6, e a classe que não existe

- Com o momentum certo, a mesma rede faz 1,000 no 54 : 0 : 6 — o cruzamento sintético é fácil
- Posta diante de 167 obstáculos, ela chamou 157 de via livre e 10 de cruzamento
- Zero obstáculos: ela não tem como responder uma classe que nunca viu
- E a validação dela não mostra isso, porque também não tem obstáculo
- Repetir os 6 exemplos raros 50 vezes não cria o que a câmera não viu: capture mais

BLOCO 4

A entrega do TP4, e a decisão que abre o TP5

O relatório do TP4: o que se declara, item por item

- Item 1: o mundo que você usou — o mínimo da Aula 11 ou o Gazebo Fortress — e por quê
- Item 2: onde mora a decisão de interromper, e o limiar que a régua leu no SEU bag
- Item 3: a troca de "placa" por cone e veículo, com a frase de que o MetaDrive não tem placa
- Item 3: a configuração do detector por fonte de imagem, declarada antes de medir
- Item 4: quantos rótulos você corrigiu, e a contagem por classe antes de treinar
- Item 4: as duas rodadas com a mesma semente, a linha de base e a matriz
- Em todos: o que rodou onde — e o que você não conseguiu rodar, dito como tal

A ordem da entrega

- Tudo na main: dev mergeado, evidências commitadas, links testados em janela anônima
- A branch entrega-tp4 igual à main, e a tag tp4 empurrada
- O Moodle: o PDF do relatório, o ZIP do código e os links — é a entrega oficial
- Bag, vídeo e modelo vão por link público ou não listado: se não abre, não existe
- A hora do prazo do TP4 é diferente da dos outros TPs: está na página, confira

O TP5 começa esta semana, com uma decisão

- Opção A: hardware — o robô de verdade percebendo e se movendo
- Opção B: o drone PX4 simulado, com missão de waypoints
- A escolha vai por escrito para o docs/decisoes.md, com a data, antes da entrega do TP4
- Com plano B declarado: o que você faz se a opção escolhida travar
- Entre o TP4 e o TP5 há poucos dias: decidir tarde é perder o TP5 por relógio

Como a decisão fica registrada

```
## AAAA-MM-DD -- TP5: opcao B (drone PX4 simulado)

- O que mudou: o TP5 sera validado em simulacao, com missao de waypoints.
- Onde: docs/decisoes.md, PROJETO.md (plano do TP5).
- Por que: sem Raspberry Pi; o projeto ja e simulado desde o TP3.
- Plano B: se o PX4 nao subir ate a data do gate, ...
- Impacto: ...
```

Data, o que mudou, onde, por quê, impacto — o formato do manual do aluno — e o plano B, que é o que o gate pede além da escolha.

A tarefa da semana: é ali que a aula acontece para você

- 1. Os terminais, e o pacote da árvore compilado no seu workspace
- 2. O seu bag: gravar, tocar com --clock, e a régua e a figura sobre ele
- 3. O MetaDrive instalado, e 500 quadros em cinco sementes
- 4. As faixas com a configuração da SUA câmera, e as formas
- 5. Rotular com revisão, contando as correções
- 6. A rede: as duas rodadas com a mesma semente, as curvas e a matriz
- 7. O relatório e a entrega; 8. a decisão do TP5; 9. como saber que terminou

Como saber que terminou

```
python3 medir_sessao.py SEU.csv              # pelo menos um evento, e o veredito lido
wc -l quadros-s*/estado.csv                  # 5 x 101 linhas: 500 quadros e os cabecalhos
awk -F, 'NR>1 && $2!=$3' quadros-s7/rotulos.csv | wc -l   # quantos voce corrigiu
cat resultado-metadrive/tabela.txt           # linha de base, as duas rodadas, a matriz
grep -c "TP5" "$PB_DIR/docs/decisoes.md"     # a decisao registrada
```

E o relatório lido com o enunciado do lado. Travou e nada na página bate? Fórum do Moodle, com a saída do comando e o passo em que você está.

Desafio: qual coluna denuncia?

- Reproduza a armadilha: a mesma rede com --bn-momento 0.99, e as curvas das duas
- Ou: as duas rodadas de novo com outra semente — a ordem 0,910 x 0,940 se mantém?
- Pergunta: qual coluna da tabela denuncia a mudança, e qual fica igual?
- A resposta cabe numa frase, e vale mais no relatório do que mais uma época de treino

Na próxima: o TP4 aterrissa, item por item.

A próxima aula lê o enunciado do TP4 com os repositórios de vocês do lado: o que cada item pede, o que a evidência precisa mostrar, e o que costuma faltar. Tragam o bag medido, os quadros rotulados e a tabela das duas rodadas — e a decisão do TP5 escrita. As datas estão todas na página e em "Gates de cada TP".